Linux Kernel Programming

Linux Kernel programming

Agenda

- Input Driver
- GPIO Driver
- GPIO Using Sysfs
- GPIO controller Driver – Gpio Chip
- GPIO Driver

Input Driver

- The Input subsystem takes care of all the input events coming from the human user.

- Input device drivers will capture the hardware event information in an uniform format (struct input_event structure) and report to the core layer, then the core layer sorts the data, escalates to the appropriate event-handling driver, and finally through the events layer passes the information to user space. Applications using /dev device node can get event information.

- Initially written to support the USB HID (Human Interface Device) devices, the Input subsystem quickly grew up to handle all kind of inputs (using USB or not): keyboards, mice, joysticks, touchscreens, etc.

- The Input subsystem is split in two parts:
  - **Device drivers:** they talk to the hardware (e.g., USB, I2C), and provide events (e.g., keystrokes, accelerometer movements, touchscreen coordinates) to the input core.

Input Driver

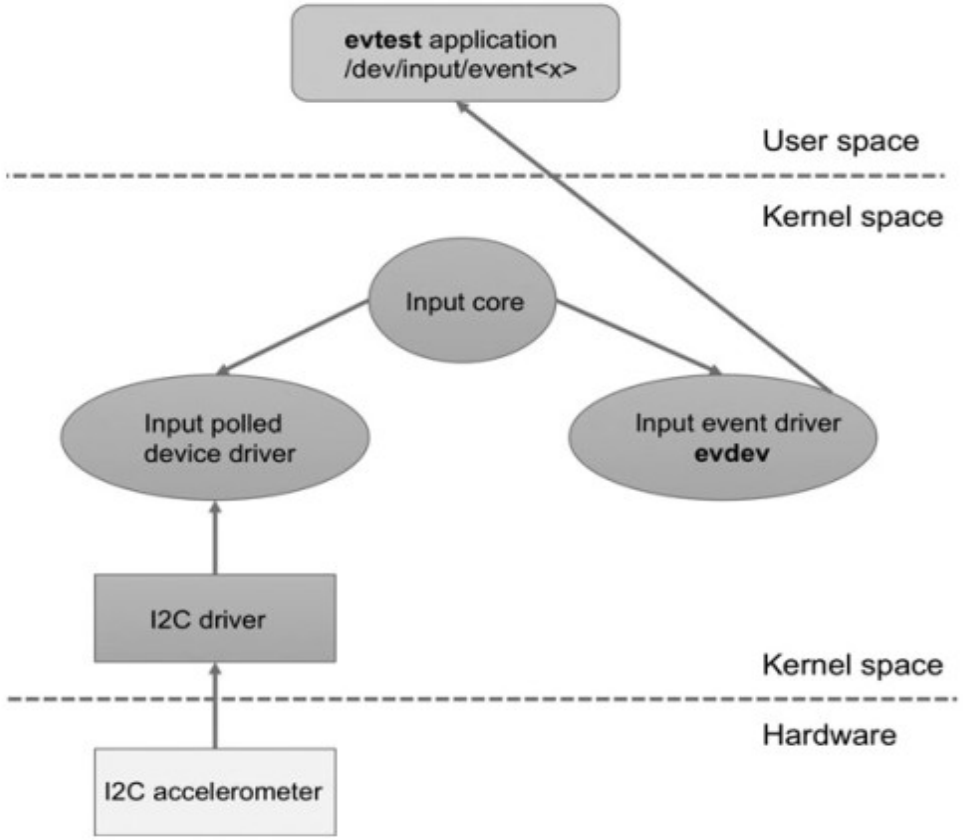
- Event handlers: Input event drivers that get events from device drivers and pass them to user space and in-kernel consumers, as needed via various interfaces. The evdev driver is a generic input event interface in the Linux kernel. It generalizes raw input events from device drivers and makes them available through character devices in the /dev/input/ directory. The event interface will represent each input device as a /dev/input/event<X> character device. This is the preferred interface for user space to consume user input, and all clients are encouraged to use it.

- You can use blocking and nonblocking reads, and also select() on the /dev/input/eventX devices, and you'll always get a whole number of input events on a read. Their layout is:

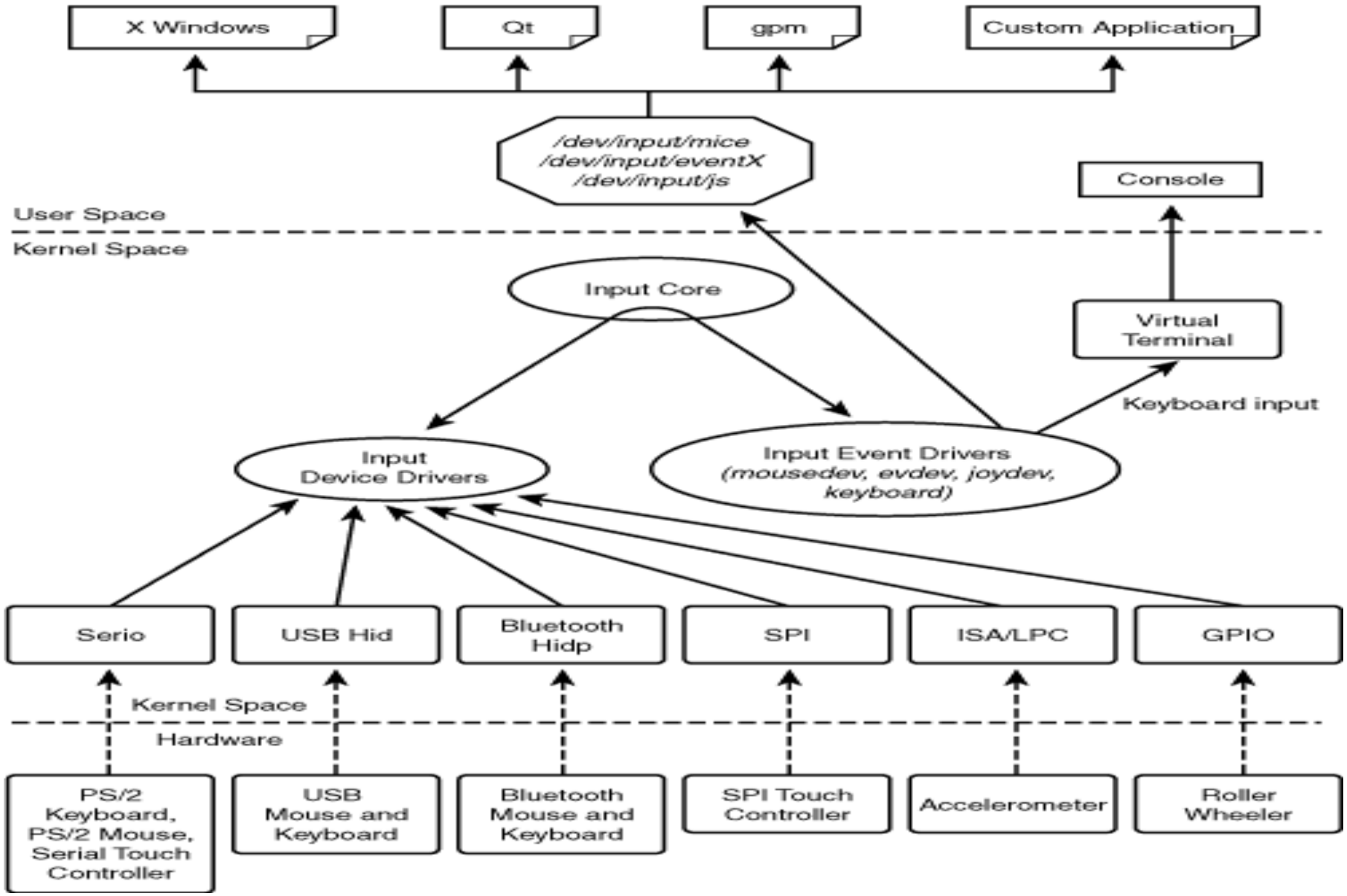
```
struct input_event {
struct timeval time;
unsigned short type;
unsigned short code;
unsigned int value;
};
```

Input Driver

- A very useful application for input device testing is evtest located at http://cgit.freedesktop.org/ evtest/. The evtest application displays information on the input device specified on the command line, including all the events supported by the device. It then monitors the device and displays all the events layer events generated.



Input Driver



Input Driver

- the main file to include, in order to interface with the input subsystem is linux/input.h:

```
#include <linux/input.h>
```

Input Driver

- No matter what type of input device it is, and what type of event it sends, an input device is represented in the kernel as an instance of struct input_dev:

```
struct input_dev {
  const char *name;
  const char *phys;
  unsigned long evbit[BITS_TO_LONGS(EV_CNT)];
  unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];
  unsigned long relbit[BITS_TO_LONGS(REL_CNT)];
  unsigned long absbit[BITS_TO_LONGS(ABS_CNT)];
  unsigned long mscbit[BITS_TO_LONGS(MSC_CNT)];
  unsigned int repeat_key;
  int rep[REP_CNT];
```

Input Driver

```
        struct input_absinfo *absinfo;
    unsigned long key[BITS_TO_LONGS(KEY_CNT)];
    int (*open)(struct input_dev *dev);
    void (*close)(struct input_dev *dev);
    unsigned int users;
    struct device dev;
    unsigned int num_vals;
    unsigned int max_vals;
    struct input_value *vals;
    bool devres_managed;
};
```

- The meanings of the preceding fields are as follows:
  - **name:** represents the name of the device.

Input Driver

- **phys:** is the physical path to the device in the system hierarchy.
- **evbit:** is a bitmap of the types of events supported by the device. Some types of area are as follows:
  - **EV_KEY** is for devices supporting sending key events (keyboards, button, and so on).
  - **EV_REL** is for devices supporting sending relative positions (the mouse, digitizers, and so on).
  - **EV_ABS** is for devices supporting sending absolute positions (joysticks, for example).

The list of events is available in the kernel source, in the include/linux/input-event-codes.h file. You can use the set_bit() macro to set the appropriate bit, depending on your input devices capabilities. Of course, a device can support more than one type of event. For example, a mouse will set both EV_KEY and EV_REL:

```
set_bit(EV_KEY, my_input_dev->evbit);
set_bit(EV_REL, my_input_dev->evbit);
```

Input Driver

- **keybit:** is for an EV_KEY type-enabled device, a bitmap of keys/buttons that this device exposes; for example, BTN_0, KEY_A, KEY_B, and so on. The complete list of keys/buttons is in the include/linux/input-event-codes.h file.
- **relbit:** is for an EV_REL type enabled device, a bitmap of relative axes for the device. For example, REL_X, REL_Y, REL_Z, REL_RX, and so on. Take a look at include/linux/input-event-codes.h for the complete list.
- **absbit:** is for an EV_ABS type enabled device and shows a bitmap of absolute axes for the device; for example, ABS_Y, ABS_X, and so on. Take a look at the same previous file for the complete list.
- **mscbit:** is for EV_MSC type enabled devices and shows a bitmap of miscellaneous events supported by the device.
- **repeat_key**: stores the keycode of the last key pressed; it is used to implement software auto-repeat.
- **rep:** is for the current values of auto-repeat parameters (delay, rate).
- **absinfo:** is an array of &struct input_absinfo elements, holding information about absolute axes (current value, min, max, flat, fuzz, resolution, and so on).

Input Driver

You should use the input_set_abs_params() function to set those values:

```
void input_set_abs_params(struct input_dev *dev, unsigned int axis,
                          int min, int max, int fuzz, int flat)
```

- **min and max:** specify the lower and upper bound values.
- **fuzz:** indicates the expected noise on the specified channel of the specified input device. The following is an example in which we only set each channel's bounds:

```
#define ABSMAX_ACC_VAL 0x01FF
#define ABSMIN_ACC_VAL -(ABSMAX_ACC_VAL)
[...]
set_bit(EV_ABS, idev->evbit);
input_set_abs_params(idev, ABS_X, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);
```

Input Driver

```
input_set_abs_params(idev, ABS_Y, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);
input_set_abs_params(idev, ABS_Z, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);
```

- **key:** reflects the current state of the device's keys/buttons.
- **open:** is a method called when the very first user calls input_open_device(). Use this method to prepare the device, including the interrupt request, the polling thread start, and so on.
- **close:** is called when the very last user calls input_close_device(). Here, you can stop polling (which consumes a lot of resources).
- **users:** stores the number of users (input handlers) that opened this device. It is used by input_open_device() and input_close_device(), to make sure that dev->open() is only called when the first user opens the device, and dev->close() is called when the very last user closes the device.
- **dev:** is the struct device associated with the device (for device model).
- **num_vals:** is the number of values queued in the current frame.

Input Driver

- **max_vals:** is the maximum number of values queued in a frame.
- **vals:** is the array of values queued in the current frame.
- **devres_managed:** indicates that devices are managed with the devres framework and need not be explicitly unregistered or freed.

Input Driver – Allocation and Registration

- Prior to registering and sending the event with an input device, it should be allocated with the **input_allocate_device()** function. In order to free the previously allocated memory for a non-registered input device, the **input_free_device()** function should be used. If the device has already been registered, **input_unregister_device()** should be used instead. Like every function where memory allocation is needed, we can use a resource-managed version of functions, as follows:

```
struct input_dev *input_allocate_device(void)
struct input_dev *devm_input_allocate_device(struct device *dev)
void input_free_device(struct input_dev *dev)
static void devm_input_device_unregister(struct device *dev,
                                         void *res)
int input_register_device(struct input_dev *dev)
void input_unregister_device(struct input_dev *dev)
```

- Device allocation may sleep, and therefore, it must not be called in the atomic context or with a spinlock held.

Input Driver – Allocation and Registration

The following is an excerpt of the probe function of an input device sitting on the I2C bus:

```
struct input_dev *idev;
int error;
idev = input_allocate_device();
if (!idev)
    return -ENOMEM;

idev->name = BMA150_DRIVER;
idev->phys = BMA150_DRIVER "/input0";
idev->id.bustype = BUS_I2C;
idev->dev.parent = &client->dev;
```

Input Driver – Allocation and Registration

```
set_bit(EV_ABS, idev->evbit);
input_set_abs_params(idev, ABS_X, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);
input_set_abs_params(idev, ABS_Y, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);
input_set_abs_params(idev, ABS_Z, ABSMIN_ACC_VAL,
                     ABSMAX_ACC_VAL, 0, 0);

error = input_register_device(idev);
if (error) {
    input_free_device(idev);
    return error;
}
```

Input Driver – Allocation and Registration

```
error = request_threaded_irq(client->irq,
        NULL, my_irq_thread,
        IRQF_TRIGGER_RISING | IRQF_ONESHOT,
        BMA150_DRIVER, NULL);
if (error) {
    dev_err(&client->dev, "irq request failed %d, error %d\n",
        client->irq, error);
    input_unregister_device(bma150->input);
    goto err_free_mem;
}
```

Input Driver – Polled input device

- A polled input device is a special type of input device, which relies on polling to sense device state changes whereas the generic input device type relies on the IRQ to sense changes and send events to the input core.

- A polled input device is described as an instance of the struct input_polled_dev structure in the kernel, and is a wrapper around the generic struct input_dev structure:

```
struct input_polled_dev {
    void *private;
    void (*open)(struct input_polled_dev *dev);
    void (*close)(struct input_polled_dev *dev);
    void (*poll)(struct input_polled_dev *dev);
    unsigned int poll_interval; /* msec */
    unsigned int poll_interval_max; /* msec */
```

Input Driver – Polled input device

```
    unsigned int poll_interval_min; /* msec */
    struct input_dev *input;
    bool devres_managed;
};
```

- The following are the meanings of the elements in this structure:
  - **private:** is the driver's private data.
  - **open:** is an optional method that prepares a device for polling (enabling the device, and maybe flushing the device state).
  - **close:** is an optional method that is called when the device is no longer being polled. It is used to put devices into low power mode.
  - **poll:** is a mandatory method called whenever the device needs to be polled. It is called at the frequency of poll_interval.
  - **poll_interval:** is the frequency at which the poll() method should be called. It defaults to 500 msec, unless overridden when registering the device.

Input Driver – Polled input device

- **poll_interval_max:** specifies the upper bound for the poll interval. It defaults to the initial value of poll_interval.
- **poll_interval_min:** specifies the lower bound for the poll interval. It defaults to 0.
- **input:** is the input device around which the polled device is built. It must be properly initialized by the driver (ID, name, and bits). A polled input device provides an interface to use polling instead of IRQ, to sense device state changes.

- Allocating/freeing the struct input_polled_dev structure is done by using **input_allocate_polled_device()** and **input_free_polled_device().** You should take care of initializing the mandatory fields of the struct input_dev embedded in it. The polling interval should be set, too; otherwise, it defaults to 500 msec. You can use resource manage version too. Both prototypes are as follows:

struct input_polled_dev *devm_input_allocate_polled_device(struct device *dev)

struct input_polled_dev *input_allocate_polled_device(void)

void input_free_polled_device(struct input_polled_dev *dev)

Input Driver – Polled input device

- After allocation and proper field initialization, the polled input device can be registered by using **input_register_polled_device(),** which returns 0 on success. The reverse operation (unregister) is done with the **input_unregister_polled_device()** function:

```
int input_register_polled_device(struct input_polled_dev *dev)
void  input_unregister_polled_device(struct input_polled_dev *dev)
```

- A typical example of the probe() function for such a device is as follows:

```
static int button_probe(struct platform_device *pdev)
{
    struct my_struct *ms;
    struct input_dev *input_dev;
    int retval;
    ms = devm_kzalloc(&pdev->dev, sizeof(*ms), GFP_KERNEL);
    if (!ms)
        return -ENOMEM;
```

Input Driver – Polled input device

```
ms->poll_dev = input_allocate_polled_device();
if (!ms->poll_dev){
    kfree(ms);
    return -ENOMEM;
}

/* This gpio is not mapped to IRQ */
ms->reset_btn_desc = gpiod_get(dev, "reset", GPIOD_IN);

ms->poll_dev->private = ms ;
ms->poll_dev->poll = my_btn_poll;
ms->poll_dev->poll_interval = 200; /* Poll every 200ms */
ms->poll_dev->open = my_btn_open; /* consist */
```

Input Driver – Polled input device

```
input_dev = ms->poll_dev->input;
input_dev->name = "System Reset Btn";

/* The gpio belong to an expander sitting on I2C */
input_dev->id.bustype = BUS_I2C;
input_dev->dev.parent = &pdev->dev;

/* Declare the events generated by this driver */
set_bit(EV_KEY, input_dev->evbit);
set_bit(BTN_0, input_dev->keybit); /* buttons */
```

Input Driver – Polled input device

```
        retval = input_register_polled_device(mcp->poll_dev);
        if (retval) {
            dev_err(&pdev->dev, "Failed to register input device\n");
            input_free_polled_device(ms->poll_dev);
            kfree(ms);
        }
        return retval;
    }
```

- The following is how our struct my_struct structure looks:

```
struct my_struct {
    struct gpio_desc *reset_btn_desc;
    struct input_polled_dev *poll_dev;
}
```

Input Driver – Polled input device

- The following is how the open function looks:

```
static void my_btn_open(struct input_polled_dev *poll_dev)
{
    struct my_strut *ms = poll_dev->private;
    dev_dbg(&ms->poll_dev->input->dev, "reset open()\n");
}
```

- The open method is used to prepare the resources needed by the device. We do not really need this method for this example.

Input Driver – Generating and Reporting

- Device allocation and registration are essential, but they are not the main goals of an input device driver, which is designed to report events to the input core. Depending on the type of event your device supports, the kernel provides the appropriate APIs to report them to the core.

- Given an EV_XXX capable device, the corresponding report function would be input_report_xxx(). The following table shows mappings between the most important event types and their report functions:

Event type	Report function	Code example
EV_KEY	input_report_key()	input_report_key(poll_dev->input, BTN_0, gpiod_get_value(ms-> reset_btn_desc) & 1);
EV_REL	input_report_rel()	input_report_rel(nunchuk->input, REL_X, (nunchuk->report.joy_x - 128)/10);

EV_ABS	input_report_abs()	input_report_abs(bma150->input, ABS_X, x_value);
		input_report_abs(bma150->input, ABS_Y, y_value);
		input_report_abs(bma150->input, ABS_Z, z_value);

Input Driver – Generating and Reporting

- Their respective prototypes are as follows:

  void input_report_abs(struct input_dev *dev, unsigned int code, int value)

  void input_report_key(struct input_dev *dev, unsigned int code, int value)

  void input_report_rel(struct input_dev *dev, unsigned int code, int value)

- The list of available report functions can be found in include/linux/input.h, in the kernel source file. They all have the same skeleton, as follows:
  - **dev:** is the input device responsible for the event.
  - **code:** represents the event code; for example, REL_X or KEY_BACKSPACE. The complete list is in include/linux/input-event-codes.h.
  - **value:** is the value the event carries. For the EV_REL event type, it carries the relative change. For a EV_ABS (joysticks, and so on) event type, it contains an absolute new value. For the EV_KEY event type, it should be set to 0 for key release, 1 for key press, and 2 for auto-repeat.

Input Driver – Generating and Reporting

- After all changes have been reported, the driver should call input_sync() on the input device, in order to indicate that this event is complete. The input subsystem will collect these into a single packet and send it through /dev/input/event<X>, which is the character device representing our struct input_dev on the system and where <X> is the interface number assigned to the driver by the input core:

void input_sync(struct input_dev *dev)

GPIO using sysfs

- The GPIOs are represented on Raspberry Pi as files and can be found in the directory tree under “/ sys / class / gpio”. This is called “sysfs”.

```
ls /sys/class/gpio
```

- When controlling and programming you have to make sure that you do not choose the pin number, but the BCM number of the GPIO. In the following example, we configure the GPIO 19 (pin 35) as the output.

- To configure a GPIO you have to create it first. Then you decide on the direction. So either as input or output. First, we turn on the “GPIO 19”.

```
echo 19> /sys/class/gpio/export
```

GPIO using sysfs

- If the GPIO is to be configured as an input, the following command applies.

  echo in> /sys/class/gpio/gpio19/direction

- If the GPIO is to be configured as an output, the following command applies.

  echo out> /sys/class/gpio/gpio19/direction

- A GPIO configured as an output should then be given a status, ie “high” or “low”.

- Basically you can only set outputs. To set the state of a GPIO output to “high”, the following command is sufficient.

  echo 1> /sys/class/gpio/gpio19/value

GPIO using sysfs

- To set the state of a GPIO output to “low” the following command is sufficient.

  echo 0> /sys/class/gpio/gpio19/value

- To determine the state, ie “high” or “low”, at a GPIO, the following command is sufficient. The GPIO can be both an input and an output.

  cat /sys/class/gpio/gpio17/value

- The output is “1” for “high” or “0” for “low”.

GPIO using sysfs

- If you no longer need GPIOs, then you should disable them. The following commands will cause the files and directories of the GPIOs to disappear. When restarting, the GPIOs are automatically reset.

echo 19> /sys/class/gpio/unexport

GPIO Controller Driver

- GPIO Controller Drivers for such devices should provide the following:
  - Methods to establish GPIO direction (input and output).
  - Methods used to access GPIO values (get and set).
  - Methods to map a given GPIO to IRQ and return the associated number.
  - A flag saying whether calls to its methods may sleep. This is very important.
  - An optional debugfs dump method (showing extra state such as pullup config).
  - An optional number called a base number, from which GPIO numbering should start. It will be automatically assigned if omitted.

GPIO Controller Driver

- In the kernel, a GPIO controller is represented as an instance of struct gpio_chip, defined in linux/gpio/driver.h:

```
struct gpio_chip {
  const char *label;
  struct device *dev;
  struct module *owner;
  int (*request)(struct gpio_chip *chip, unsigned offset);
  void (*free)(struct gpio_chip *chip, unsigned offset);
  int (*get_direction)(struct gpio_chip *chip, unsigned offset);
  int (*direction_input)(struct gpio_chip *chip, unsigned offset);
  int (*direction_output)(struct gpio_chip *chip, unsigned offset,
        int value);
```

GPIO Controller Driver

```
        int (*get)(struct gpio_chip *chip,unsigned offset);
void (*set)(struct gpio_chip *chip, unsigned offset, int value);
void (*set_multiple)(struct gpio_chip *chip, unsigned long *mask,
        unsigned long *bits);
int (*set_debounce)(struct gpio_chip *chip, unsigned offset,
        unsigned debounce);

int (*to_irq)(struct gpio_chip *chip, unsigned offset);

int base;
u16 ngpio;
const char *const *names;
bool can_sleep;
bool irq_not_threaded;
bool exported;
```

GPIO Controller Driver

```
#ifdef CONFIG_GPIOLIB_IRQCHIP
  /*
   * With CONFIG_GPIOLIB_IRQCHIP we get an irqchip
    * inside the gpiolib to handle IRQs for most practical cases.
   */
  struct irq_chip *irqchip;
  struct irq_domain *irqdomain;
  unsigned int irq_base;
  irq_flow_handler_t  irq_handler;
  unsigned int irq_default_type;
#endif
```

GPIO Controller Driver

```
#if defined(CONFIG_OF_GPIO)
  /*
   * If CONFIG_OF is enabled, then all GPIO controllers described in the
   * device tree automatically may have an OF translation
   */
  struct device_node *of_node;
  int of_gpio_n_cells;
  int (*of_xlate)(struct gpio_chip *gc,
      const struct of_phandle_args *gpiospec, u32 *flags);
#endif
}
```

GPIO Controller Driver

- The following is the meaning of each element in the structure:
  - **request** is an optional hook for chip-specific activation. If provided, it is executed prior to allocating GPIO whenever you call gpio_request() or gpiod_get().
  - **free** is an optional hook for chip-specific deactivation. If provided, it is executed before the GPIO is deallocated whenever you call gpiod_put() or gpio_free().
  - **get_direction** is executed whenever you need to know the direction of the GPIO offset. Return value should be 0 to mean out, and 1 to mean in(the same as GPIOF_DIR_XXX), or negative error.
  - **direction_input** configures the signal offset as input, or returns an error.
  - **get** returns the value of GPIO offset; for output signals, this returns either the value actually sensed or zero.
  - **set** assigns an output value to the GPIO offset.
  - **set_multiple** is called when you need to assign output values for multiple signals defined by mask. If not provided, the kernel will install a generic hook that will walk through mask bits and execute chip->set(i) on each bit set.

GPIO Controller Driver

- **set_debounce** if supported by the controller, this hook is an optional callback provided to set the debounce time for the specified GPIO.
- **to_irq** is an optional hook to provide GPIO to IRQ mapping. This is called whenever you want to execute the gpio_to_irq() or gpiod_to_irq() function. This implementation may not sleep.
- **base** identifies the first GPIO number handled by this chip; or, if negative during registration, the kernel will automatically (dynamically) assign one.
- **ngpio** is the number of GPIOs this controller provides; it starts from base to (base + ngpio - 1).
- **names**, if set, must be an array of strings to use as alternative names for the GPIOs in this chip. The array must be ngpio sized, and any GPIO that does not need an alias may have its entry set to NULL in the array.
- **can_sleep** is a Boolean flag to be set if the get()/set() method may sleep. It is the case for the GPIO controller (also known as an expander) sitting on a bus, such as I2C or SPI, whose access may lead to sleep. This implies that, if the chip supports IRQs, these IRQs need to be threaded as the chip access may sleep when, for example, reading out the IRQ status registers. For a GPIO controller mapped to memory (part of SoC), this can be set to false.
- **irq_not_threaded** is a Boolean flag and must be set if can_sleep is set, but the IRQs don't need to be threaded.

GPIO Controller Driver

- After every callback has been defined and other fields set, you should call gpiochip_add() on the configured struct gpio_chip structure in order to register the controller with the kernel. When it comes to unregistering, use gpiochip_remove().

GPIO Driver

- Already Kernel GPIO subsystems have provided the APIs to access the GPIOs.

```
#include <linux/gpio.h>
```

GPIO Driver - Validate the GPIO

- Before using the GPIO, we must check the GPIO that we are planning to use is valid or not. To do that we have to use the below API.

bool gpio_is_valid(int gpio_number);

Where,

gpio_number : GPIO that you are planning to use

- It returns false if it is not valid otherwise, it returns true.

- This API determines whether the GPIO number is valid or not. Sometimes this call will return false even if you send a valid number. Because that GPIO pin might temporarily unused on a given board.

GPIO Driver - Request the GPIO

- Once you have validated the GPIO, then you can request the GPIO using the below APIs.

int gpio_request(unsigned gpio, const char *label)

where,

gpio : GPIO that you are planning to use.

label : label used by the kernel for the GPIO in sysfs. You can provide any string that can be seen in /sys/kernel/debug/gpio. Do cat /sys/kernel/debug/gpio. You can see the GPIO assigned to the particular GPIO.

- It returns 0 in success and a negative number in failure.

Note: GPIOs must be allocated before use.

GPIO Driver - Request the GPIO

- There are other variants also available. You can use any one of them based on your need.

int gpio_request_one(unsigned gpio, unsigned long flags, const char *label); – Request one GPIO.

int gpio_request_array(struct gpio *array, size_t num); – Request multiple GPIOs.

GPIO Driver - Export the GPIO

- For debugging purposes, you can export the GPIO which is allocated using the gpio_request()to the sysfs using the below API.

int gpio_export(unsigned int gpio, bool direction_may_change);

where,

gpio : GPIO that you want to export.

direction_may_change : This parameter controls whether user space is allowed to change the direction of the GPIO. true – Can change, false – Can't change.

- Returns zero on success, else an error.

- Once you export the GPIO, you can see the GPIO in /sys/class/gpio/*. There you can change the GPIO's value, etc.

GPIO Driver – Unexport the GPIO

- If you have exported the GPIO using the gpio_export(), then you can unexport this using the below API.

void gpio_unexport(unsigned int gpio)

Where,

gpio : GPIO that you want to unexport

GPIO Driver – Set the direction of the GPIO

- When you are working on the GPIOs, you have set the GPIO as output or input. The below APIs are used to achieve that.

- This API is used to set the GPIO direction as input.

  int gpio_direction_input(unsigned gpio)

Where,

gpio : GPIO that you want to set the direction as input.

- Returns zero on success, else an error.

GPIO Driver – Set the direction of the GPIO

- This API is used to set the GPIO direction as output.

  int gpio_direction_output(unsigned gpio, int value)

Where,

gpio : GPIO that you want to set the direction as output.

value : The value of the GPIO once the output direction is effective.

- Returns zero on success, else an error.

GPIO Driver – Change the GPIO value

- Once you set the GPIO direction as an output, then you can use the below API to change the GPIO value.

gpio_set_value(unsigned int gpio, int value);

where,

gpio : GPIO that you want to change the value.

value : value to set to the GPIO. 0 – Low, 1 – High.

GPIO Driver – Read the GPIO value

- You can read the GPIO's value using the below API.

```
int  gpio_get_value(unsigned gpio);
```

where,

gpio : GPIO that you want to read the value.

- It returns the GPIO's value. 0 or 1.

GPIO Driver – Release the GPIO

- Once you have done with the GPIO, then you can release the GPIO which you have allocated previously. The below API help in that case.

void gpio_free(unsigned int gpio);

where,

gpio : GPIO that you want to release.

GPIO Driver – Release the GPIO

- There are other variants also available. You can use any one of them based on your need.

void gpio_free_array(struct gpio *array, size_t num); – Release multiple GPIOs.

GPIO Driver with Interrupt

- Accessing the input GPIO in Linux Kernel with interrupt enabled:
  - Verify the GPIO is valid or not.
  - If it is valid, then you can request the GPIO from the Kernel GPIO subsystem.
  - Set the direction of the GPIO as an input
  - Set the debounce-interval
  - Read the GPIO.
  - You enable IRQ also for edge/level triggered if you need it.
  - Then release the GPIO while exiting the driver or once you are done.

GPIO Driver with Interrupt

Set the debounce-interval

- The below API is used for sets debounce time for a GPIO.

  int gpiod_set_debounce(unsigned gpio, unsigned debounce);

where,

gpio : GPIO that you want to set the debounce value.

debounce : Delay of the debounce.

- It returns 0 on success. otherwise, <0.

GPIO Driver with Interrupt

Get the IRQ number for the GPIO

- Using the below API, you can get the IRQ number for the specific GPIO.

int gpio_to_irq(unsigned gpio);

where,

gpio : GPIO that you want to get the IRQ number.

GPIO Driver with Interrupt

Request the IRQ

- Here where you need to register the GPIO IRQ number and its handler to the Linux Interrupts with the proper interrupt flags. You can find the details regarding requesting the GPIO here. Please note that you have to free the IRQ once you are done with the interrupt.

Interrupt Flags

- While registering the GPIO interrupt using request_irq() you can use any one of the flags based on your need.

  - IRQF_TRIGGER_RISING
  - IRQF_TRIGGER_FALLING
  - IRQF_TRIGGER_HIGH
  - IRQF_TRIGGER_LOW

Thank you